

Proyecto de ampliación de entorno app de reserva de mesas

Autor:

Robert Avelino Brito de Canha

Tutor:

Andres Chillòn Sesma

2 de junio del 2026

Índice

Resumen	3
Introducción	4
Requisitos	5
Requisitos funcionales:	5
Requisito no funcionales:	5
Prototipo/mockup	6
Diagrama de casos de usos	10
Diagrama de clase de análisis	11
Diagrama entidad-relación	12
Diagrama de clases de diseño	12
Fase testing	13
Caja blanca:	13
Caja Negra	18
Pruebas con junit:	19
Conclusión	23
Bibliografía	24

Resumen

Este proyecto surge ante la necesidad de ofrecer una alternativa eficiente y accesible en el sector de la hostelería, donde las soluciones actuales, como CoverManager o TheFork, presentan barreras económicas o un exceso de complejidad técnica que dificulta la gestión en restaurantes pequeños o emergentes. El objetivo principal es el desarrollo de una aplicación de reservas diseñada bajo una filosofía de simplicidad y agilidad, permitiendo a los usuarios centrarse exclusivamente en la gestión eficiente de su negocio.

El sistema se estructura en torno a dos perfiles de usuario claramente diferenciados: el cliente, con un acceso directo y sin fricciones para realizar sus reservas; y el administrador, quien dispone de un panel seguro para la gestión integral de las mismas. A través del análisis funcional y el modelado de casos de uso, se han implementado funcionalidades clave como el control de duplicados, asegurando la integridad de las reservas, y herramientas de administración directa que permiten modificar, eliminar y exportar reportes en formato PDF.

La arquitectura propuesta prioriza requisitos no funcionales críticos, destacando un diseño *responsive* optimizado para dispositivos móviles —canal principal de interacción de los clientes— y un robusto sistema de seguridad que garantiza la privacidad de los datos mediante protocolos de autenticación estándar. En conclusión, este desarrollo demuestra que es posible cubrir las necesidades operativas reales de un establecimiento de hostelería con una herramienta intuitiva y directa, eliminando las funcionalidades superfluas que complican la experiencia del usuario, ofreciendo así una alternativa competitiva que equilibra funcionalidad, seguridad y facilidad de uso.

Introducción

He escogido realizar el proyecto de reservas de un restaurante porque hoy en día, en el sector de la hostelería, solo existen dos alternativas conocidas y funcionales. Una de ellas es CoverManager, cuyo alto precio mensual no permite que los restaurantes pequeños o emergentes puedan permitírselo. La otra opción es TheFork que, aunque ofrece varios planes que podrían resultar más accesibles, requiere gestionar una aplicación complicada y con un exceso de funcionalidades. Por ello, me decanto por desarrollar una app de reservas sencilla y fácil de usar, que cumpla estrictamente con su tarea principal: reservar y gestionar reservas de manera ágil. De este modo, el personal de bares o restaurantes que busque facilidad y comodidad dispondrá de una opción directa, sin la necesidad de lidiar con herramientas sobrecargadas.

Requisitos

Requisitos funcionales:

- Selección de perfil:** mostrar una pantalla inicial con dos opciones claras de acceso “administrador y cliente”.
- Autenticación de administrador :** Si el usuario selecciona la opción de administrador, el sistema debe solicitar un usuario y contraseña.
- Acceso directo de cliente :** Si el usuario selecciona la opción de cliente, debe permitirle ingresar directamente sin login al área de reservas.
- **Crear reservar:** El sistema debe permitir al cliente y al administrador rellenar un formulario(fecha, hora, número de personas, nombre y contacto) para reservar una mesa.
- **Modificar reserva:** Si entra como administrador tiene que poder modificar las reservas y poder eliminarlas.
- Eliminar reservas:** El sistema debe permitir al administrador cancelar o borrar cualquier reserva del listado.
- Reportes:** Se debe permitir al administrador exportar el listado de reservas en formato pdf.
- Control de duplicado:** Se debe garantizar que no se dupliquen los datos, evitando que haya reservas en la misma mesa a la misma hora.

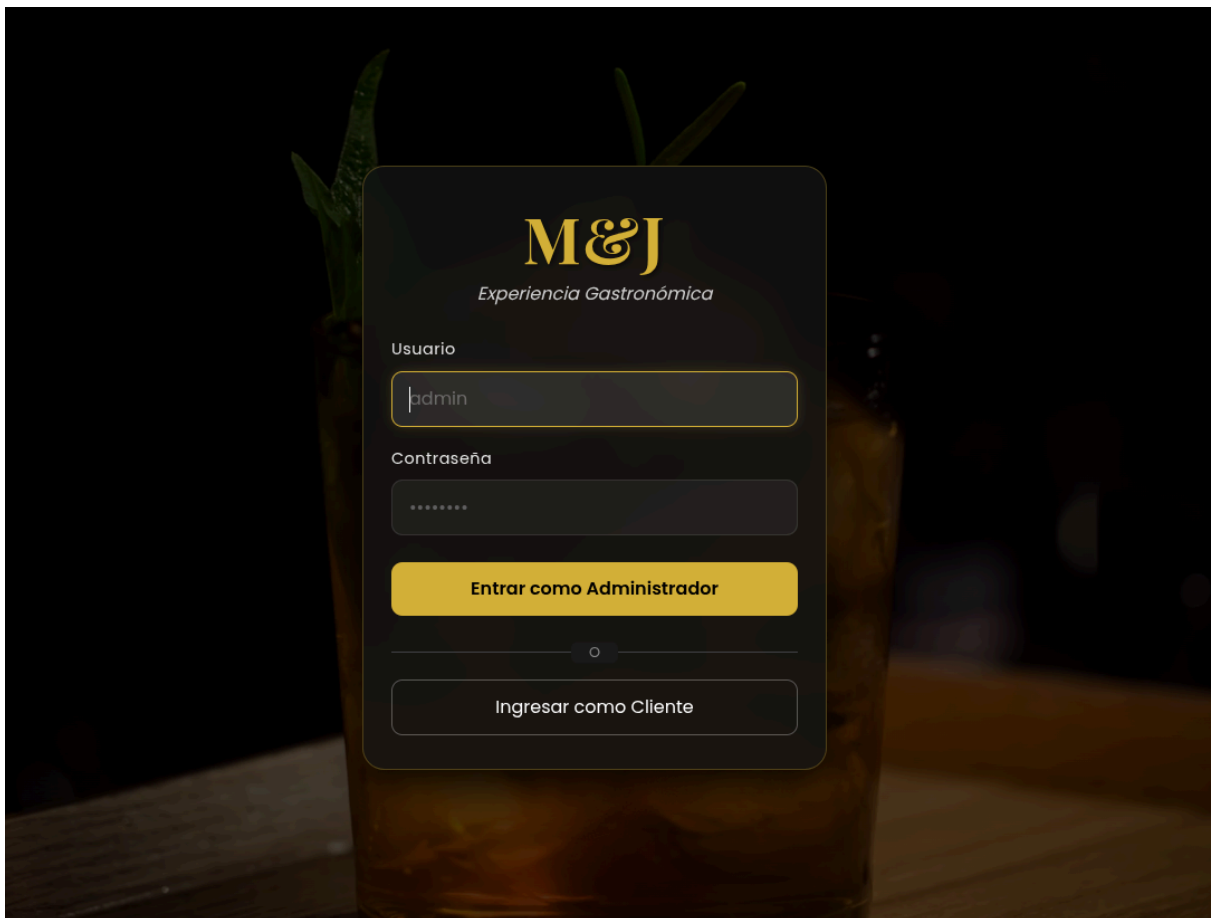
Requisito no funcionales:

- Diseño responsive:**La interfaz de reservas se debe realizar pensando en móviles y tablet, ya que la mayoría de clientes reservaran mediante un móvil.
- Privacidad de datos:** los datos que se inserten a la hora de realizar una reserva no deben verse públicamente, solamente lo podrá ver el administrador.

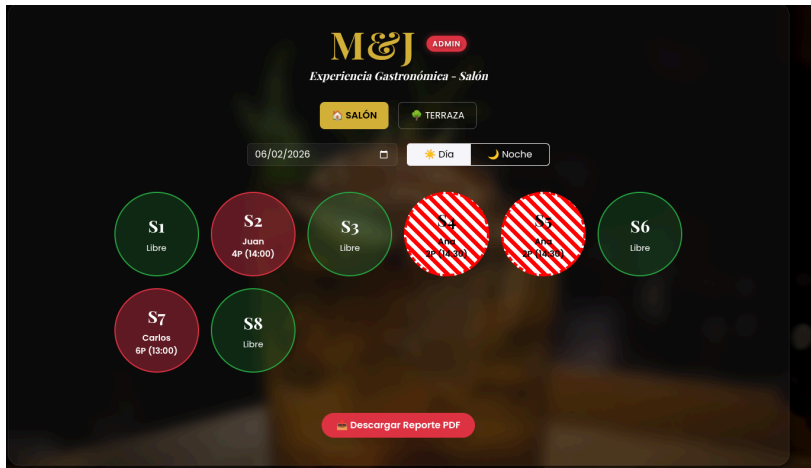
- **Seguridad de acceso:** El panel de gestión debe estar protegido contra accesos no autorizados mediante protocolos estándar de autenticación.

Prototipo/mockup

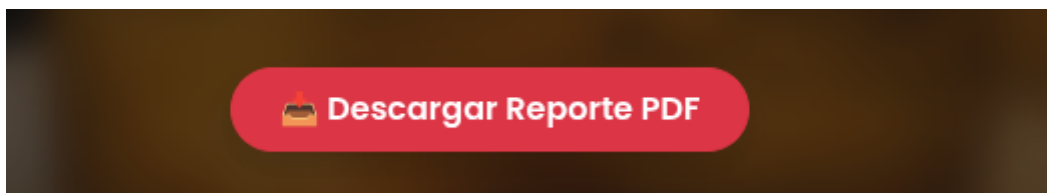
Vista login donde el administrador podrá loguearse o el cliente podrá entrar como cliente.



Si nos logueamos y entramos como administrador podemos ver las reservas. además de ver si una reserva son grupos o no. si es un grupo saldrá con un fondo blanco y rayas, si es una reserva única saldrá solo en rojo.



Además el administrador tendrá la opción de descargar reporte pdf, que nos permitirá tener las reservas en papel esto es útil para el camarero que quiera anotar a mano cosas referentes de la reserva o guiarse a su manera.



Si el administrador quiere editar una reserva solo bastará con tocar la mesa que esté en rojo o sea un grupo, esto mostrará un modal con formulario.este tendrá dos botones guardar cambios y eliminar reserva.

Experiencia Gastronómica - Salón

Detalle - Mesa S2

Nombre: Juan Apellido: Pérez

Teléfono: 555-1234

Pax: 4 Hora: 02:00 PM

Comentarios: Alergia al gluten

Guardar Cambios

Eliminar Reserva

Descargar Reporte PDF

El administrador podrá reservar mesa, tiene que seleccionar la mesa y saldra un boton para reservar.

Experiencia Gastronómica - Salón

✓ RESERVAR GRUPO: S3

Reserva - Mesas: S3

Nombre: Surname:

Pax: 2 Hora: --:--

Confirmar Reserva

S1 Libre

S2 4P (14:00)

S3 4P (14:00)

S4 4P (14:00)

S5 4P (14:00)

S6 Libre

S7 Carlos 6P (13:00)

La vista de cliente es la misma cambiando que no podrá modificar y no saldrá los datos de las otras reservas, teniendo solo una función y es reservar.

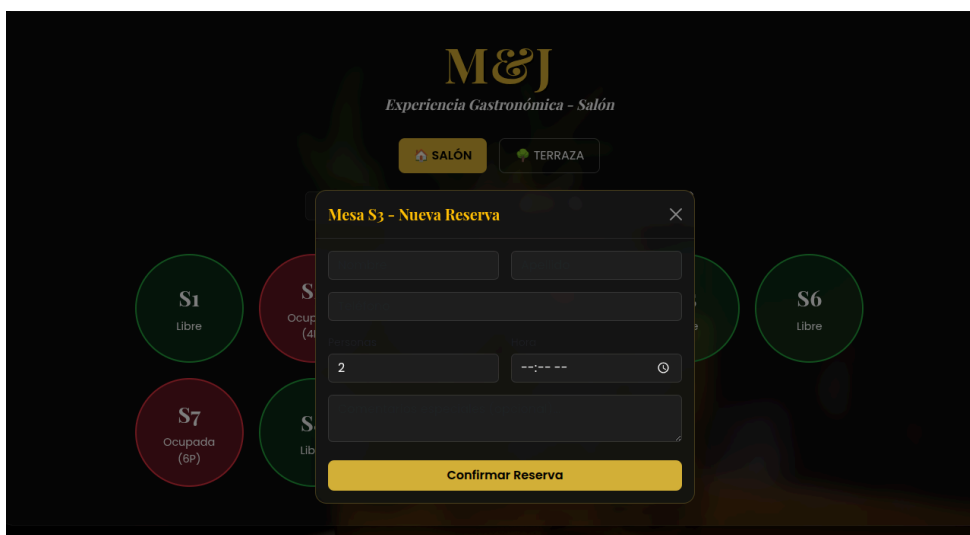
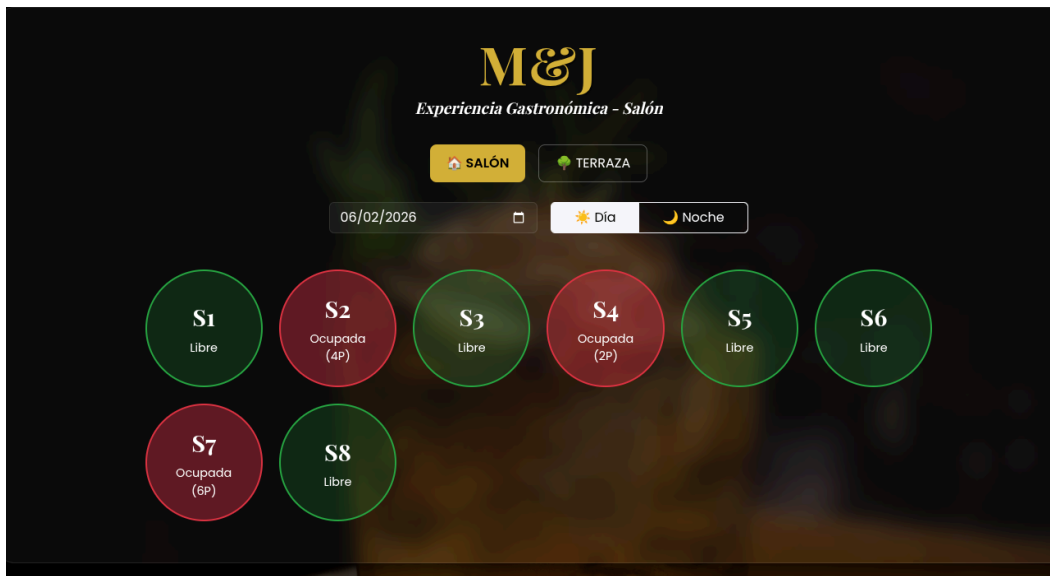


Diagrama de casos de usos

El cliente puede crear reservas y esta incluye un control de duplicados. en el caso de administrador, este inicia sesión e incluye una vista gestionar reservas donde el administrador puede eliminar reservas, modificar , exportarlas a pdf y puede crear reserva

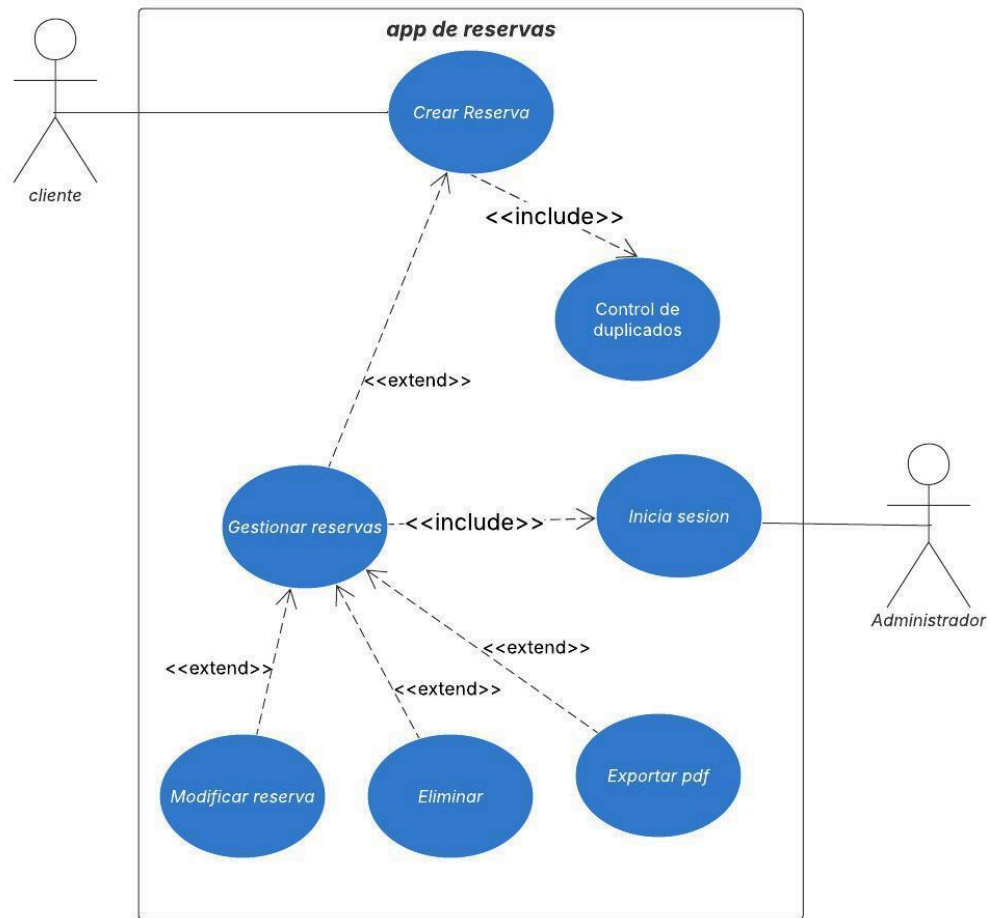


Diagrama de clase de análisis

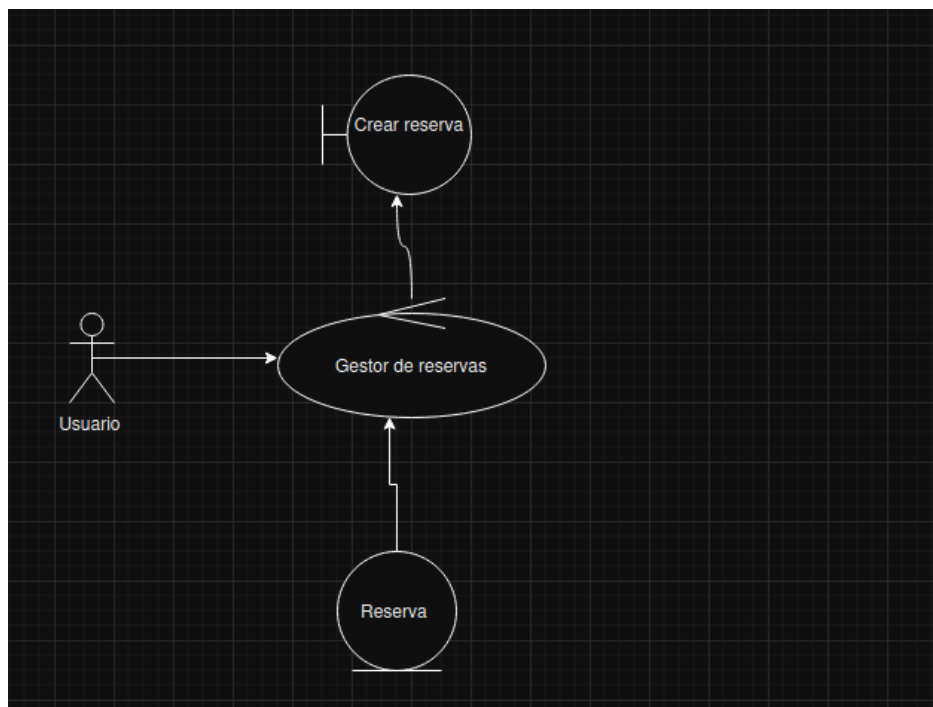
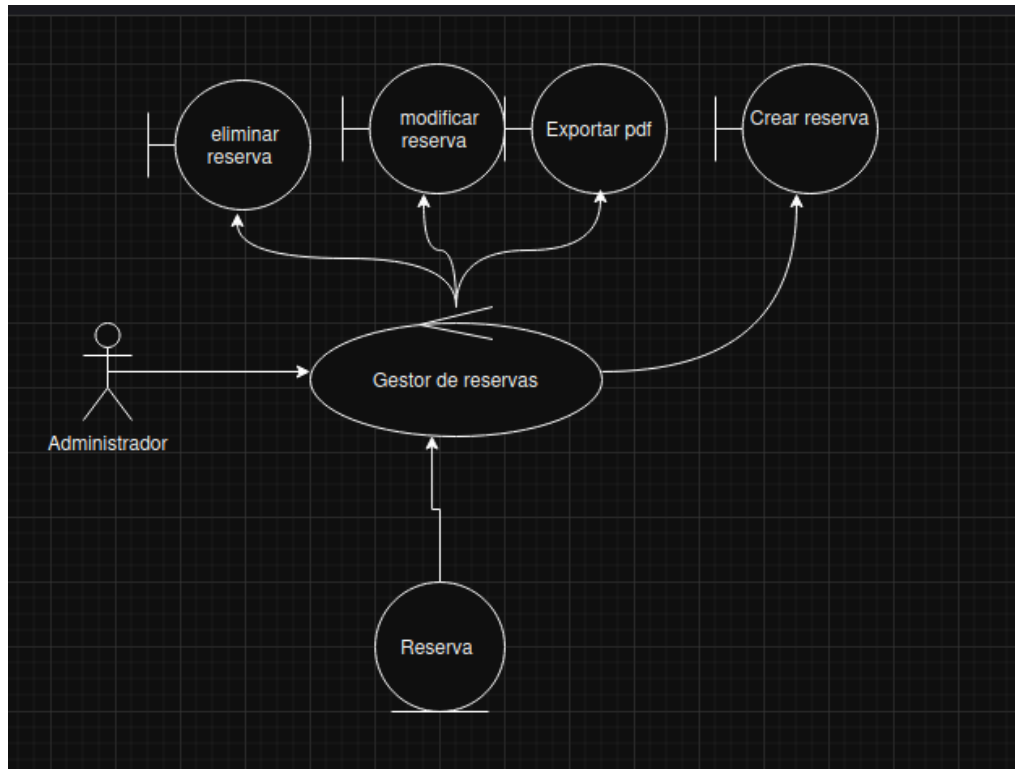


Diagrama entidad-relación

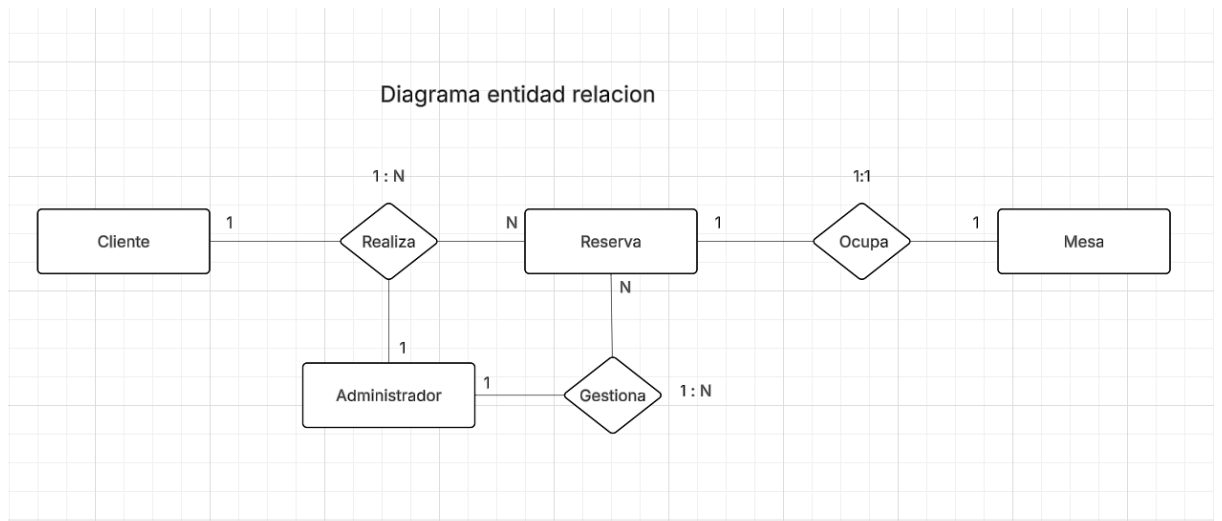
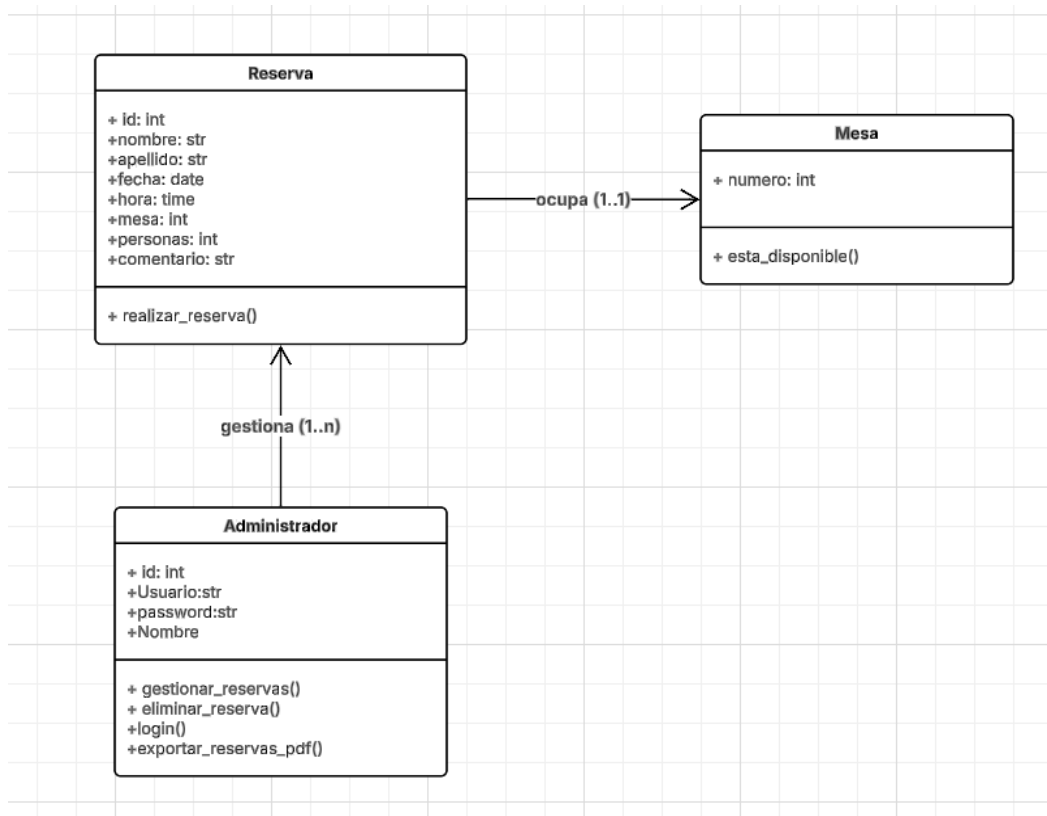


Diagrama de clases de diseño



Fase testing

Caja blanca:

Para garantizar la calidad y robustez de la funcionalidad principal, se ha realizado un análisis de **Caja Blanca** sobre la función reservar. El objetivo es verificar la correcta gestión de los datos y el control de integridad (evitar duplicados).

```

def reservar(datos):
    # 1. Validar campos obligatorios
    if campos_incompletos(datos):
        return "Error: Campos requeridos faltantes"

    # 2. Verificar disponibilidad (Control de duplicados)
    if consultar_disponibilidad(datos.fecha, datos.hora,
datos.mesa):
    # 3. Guardar reserva
  
```

```
guardar_en_bd(datos)
return "Reserva confirmada con éxito"
else:
    # 4. Error por ocupación
    return "Error: La mesa ya está reservada en ese horario"
```

La estructura lógica se ha descompuesto en 8 nodos, analizando sus tres posibles caminos

Nodo 1: Inicio de la función **reservar**.

Nodo 2: Evaluación: ¿Están los campos incompletos?

Nodo 3: Retorno: Error de validación (Campos).

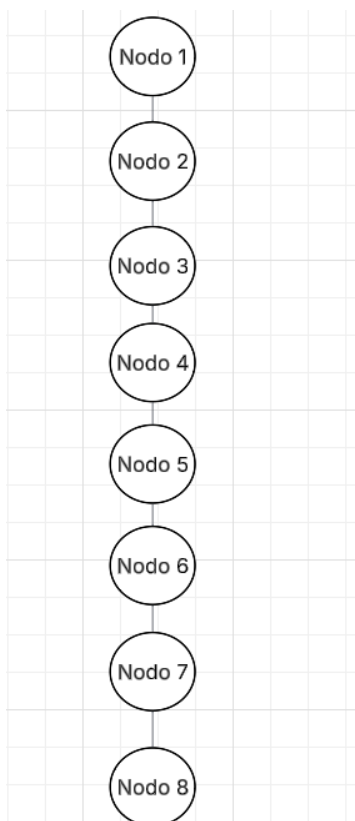
Nodo 4: Evaluación: ¿Está la mesa disponible?

Nodo 5: Ejecución: Guardar en base de datos.

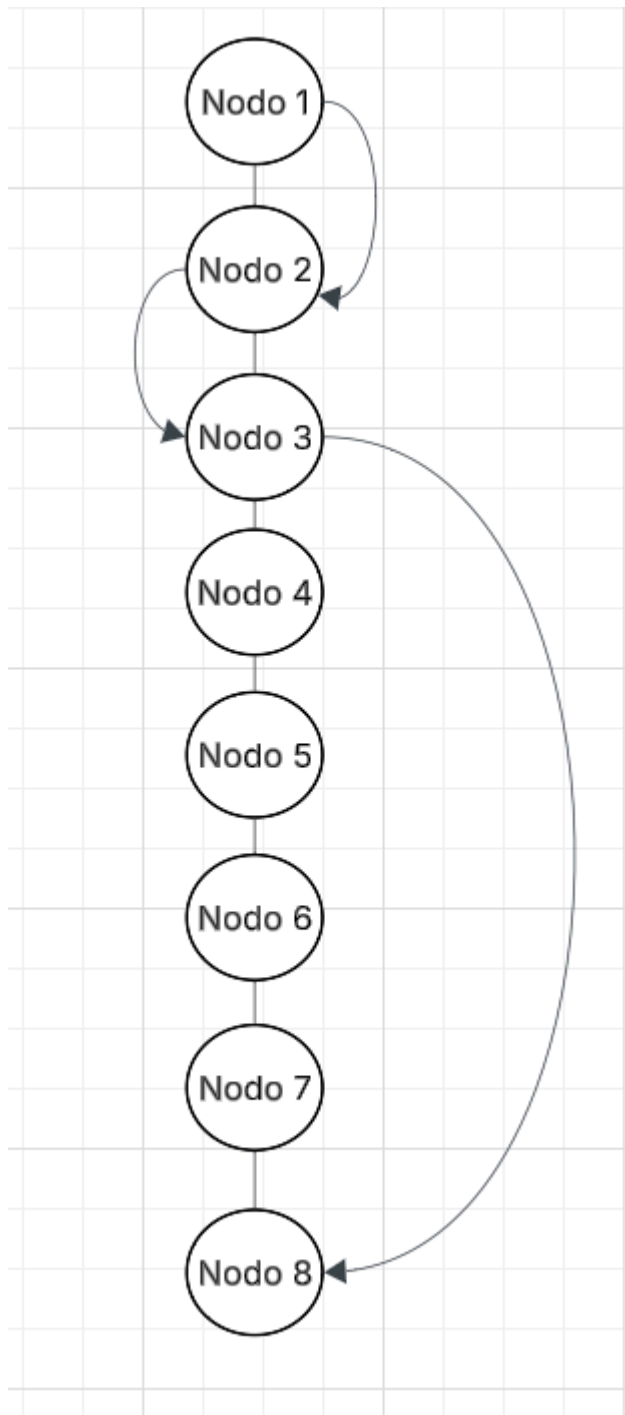
Nodo 6: Retorno: Éxito.

Nodo 7: Retorno: Error (Mesa ocupada).

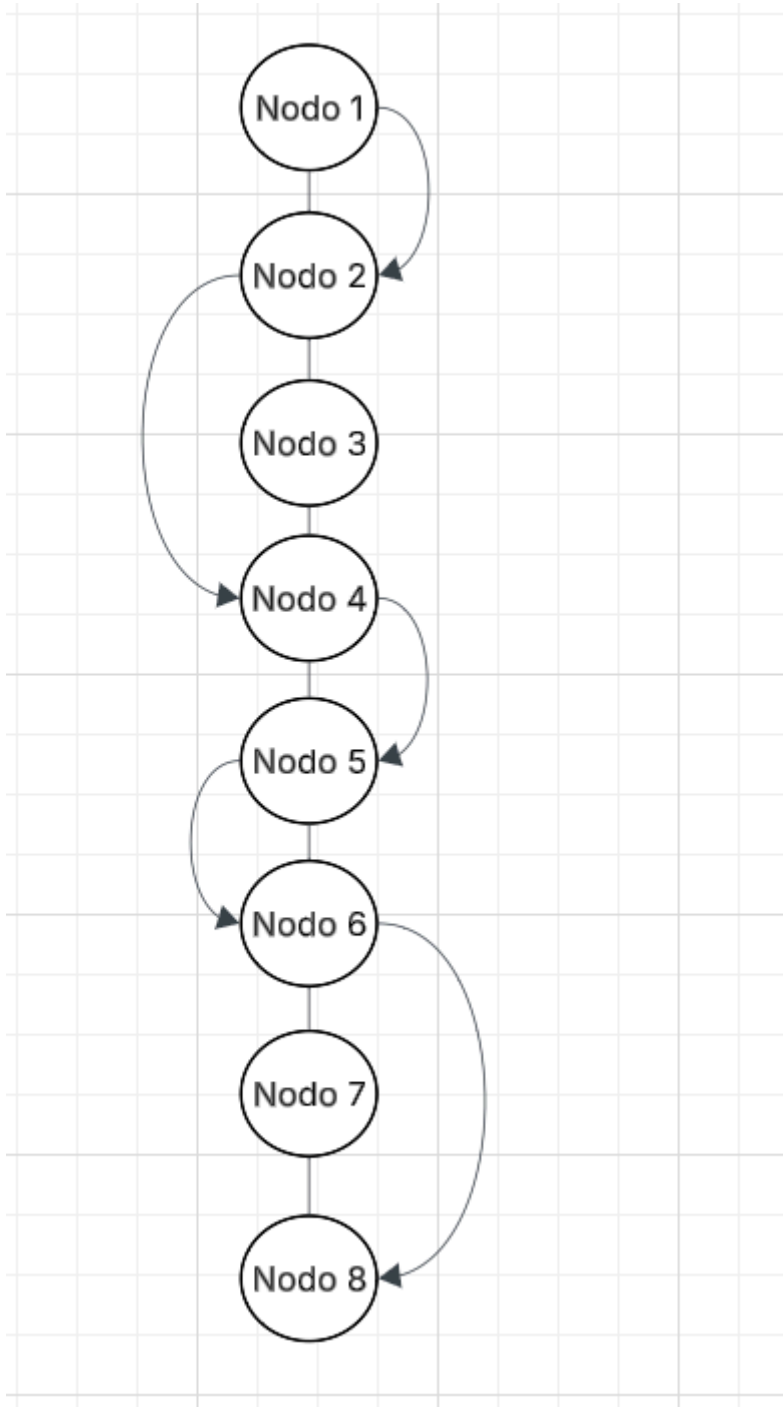
Nodo 8: Fin de la función.



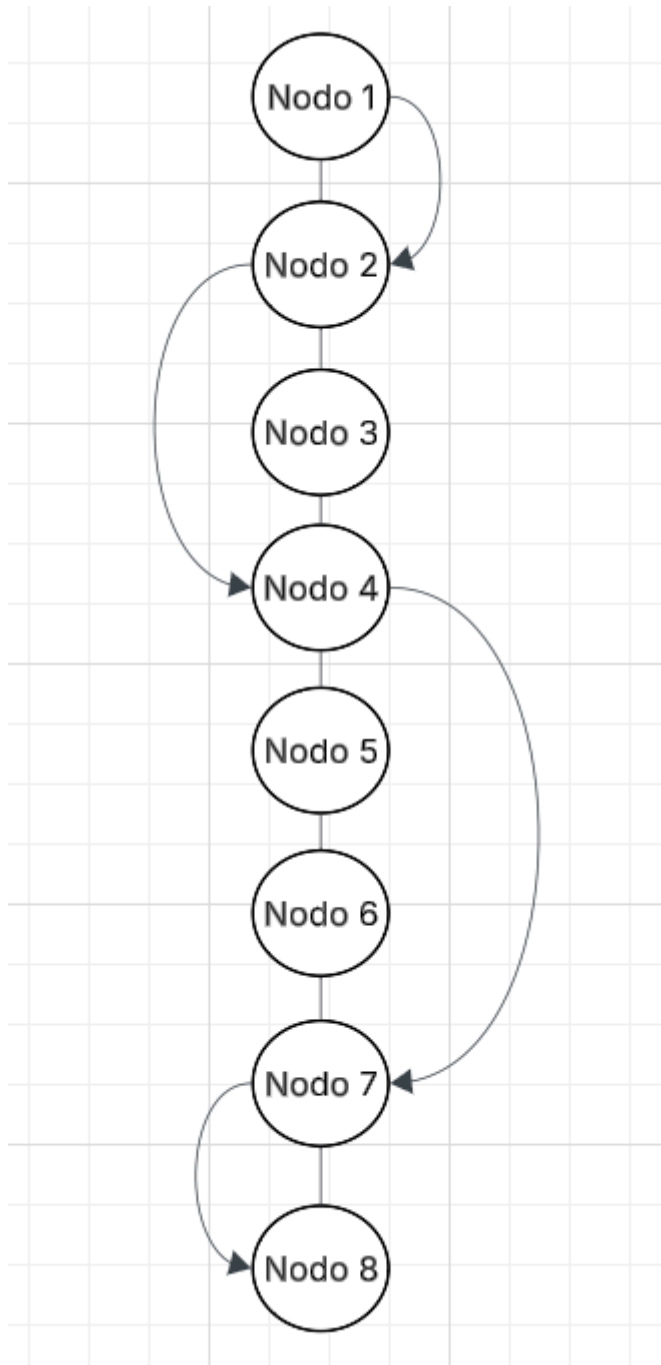
Camino 1 (error de datos) 1->2->3->8



Camino 2 (Reserva exitosa) 1->2->4->5->6->8



Camino 3 (Mesa ocupada) 1->2->4->7->8



Caja Negra

Caso de uso	Clase de Equivalencia	Descripción	Resultado esperado
Crear Reserva	CE1 (Válida)	Datos correctos, mesa libre.	Reserva guardada exitosamente.
	CE2 (Inválida)	Conflicto de horario y mismo nombre de cliente (Duplicado)	El sistema bloquea la acción.
	CE3 (Inválida)	Error en los datos	El sistema bloquea la acción.
Gestionar Reservas	CE4 (Válida)	Acceso tras login correcto.	Visualización del listado.
	CE5 (Inválida)	Acceso sin login.	Redirección a login o error.
Modificar Reserva	CE6 (Válida)	Selección de reserva existente.	Actualización de datos en BD.
Eliminar Reserva	CE7 (Válida)	Selección de reserva existente.	Eliminación del registro.
Exportar PDF	CE8 (Válida)	Generación con datos.	Descarga del archivo PDF.

Inicia Sesión	CE9 (Válida)	Credenciales correctas.	Acceso al panel de gestión.
	CE10 (Inválida)	Credenciales incorrectas.	Acceso denegado.

Pruebas con junit:

Primero creamos nuestra clase con su función de reserva y su función de comprobación duplicado

```
public class Reserva {
    private int mesa;
    private String fecha, hora;
    private String nombreCliente;

    public Reserva(String nombre, String fecha, String hora, int
mesa) {
        this.nombreCliente = nombre;
        this.fecha = fecha;
        this.hora = hora;
        this.mesa = mesa;
    }

    // Método principal de negocio
    public boolean realizarReserva(Reserva[] reservasExistentes) {
        for (Reserva r : reservasExistentes) {
            if (esMismaReserva(r)) {
                return false; // Conflicto detectado
            }
        }
    }
}
```

```

        return true; // Disponible
    }

    private boolean esMismaReserva(Reserva otra) {
        return otra != null &&
            this.fecha.equals(otra.fecha) &&
            this.hora.equals(otra.hora) &&
            this.mesa == otra.mesa;
    }
}

```

Y luego creamos nuestro archivo donde realizaremos el test insertando un primer dato y luego insertando el mismo dato a ver si detecta duplicidad. primero tenemos una función donde verificará si se reserva aplicando la función de nuestra clase reserva y su función. Luego tendremos una función (testNoPermitirReservaDuplicada) donde verificaremos que pasa si se insertan los mismos datos, lo usamos a través de la misma función que realiza la reserva y una vez que vea que está duplicada debería de evitar la inserccion

```

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertTrue;
import static org.junit.jupiter.api.Assertions.assertFalse;

public class ReservaTest {

    @Test
    public void testReservaExitosa() {
        Reserva[] baseDatosVacía = new Reserva[1];
        Reserva nuevaReserva = new Reserva("Juan", "2026-06-20",
"14:00", 5);

        assertTrue(nuevaReserva.realizarReserva(baseDatosVacía));
    }
}

```

```

@Test
public void testNoPermitirReservaDuplicada() {
    // Simulamos una mesa 5 ya ocupada a las 14:00
    Reserva[] baseDatos = { new Reserva("Ana", "2026-06-20",
"14:00", 5) };

    // Intentamos reservar la misma mesa a la misma hora
    Reserva intentoDuplicado = new Reserva("Juan",
"2026-06-20", "14:00", 5);

    assertFalse(intentoDuplicado.realizarReserva(baseDatos),
        "El sistema debería bloquear la reserva si la mesa ya
está ocupada.");
}
}

```

una vez hecho compilamos

```
javac -cp "junit-platform-console-standalone-1.10.0.jar" Reserva.java ReservaTest.java
```

y ejecutamos el test

```
java -jar junit-platform-console-standalone-1.10.0.jar --class-path . --scan-classpath
```

Una vez ejecutado vemos el resultado

```
rbrito@eno01:~/Desktop/mockup/test$ java -jar junit-platform-console-standalone-1.10.0.jar --class-path . --scan-classpath
♥ Thanks for using JUnit! Support its development at https://junit.org/sponsoring

├─ JUnit Jupiter ✓
│   └─ ReservaTest ✓
│       ├── testReservaExitosa() ✓
│       └─ testNoPermitirReservaDuplicada() ✓
├─ JUnit Vintage ✓
└─ JUnit Platform Suite ✓

Test run finished after 55 ms
[      4 containers found      ]
[      0 containers skipped    ]
[      4 containers started    ]
[      0 containers aborted    ]
[      4 containers successful  ]
[      0 containers failed     ]
[      2 tests found           ]
[      0 tests skipped         ]
[      2 tests started         ]
[      0 tests aborted         ]
[      2 tests successful      ]
[      0 tests failed          ]

WARNING: Delegated to the 'execute' command.
This behaviour has been deprecated and will be removed in a future release.
Please use the 'execute' command directly.
rbrito@eno01:~/Desktop/mockup/test$
```

GITHUB : https://github.com/robertbrito1/proyecto_ampliacion_entornos

Conclusión

En definitiva, este proyecto nace de una realidad clara: muchos restaurantes pequeños no necesitan herramientas complejas ni costosas para gestionar sus mesas, sino algo que simplemente funcione y sea fácil de usar.

Con esta aplicación, he logrado demostrar que es posible ofrecer una solución ágil y profesional que elimina las complicaciones de las grandes plataformas actuales. El resultado es un sistema que ahorra tiempo al hostelero y facilita la vida al cliente, demostrando que la tecnología es mucho más valiosa cuando se enfoca en resolver necesidades reales sin rodeos. En resumen, una herramienta práctica, directa y pensada para que los hosteleros puedan centrarse en lo único que importa: atender bien a sus clientes.

Bibliografía

<https://youtu.be/iFcDoP6jEeE?si=bddLjloQkdj5zN4A> . Este video me ayudó a tener una noción de lo que es el caso de uso, me ayudó a realizar el diagrama de caso de uso.

<https://youtu.be/OfCiwQLfsGM?si=UvQRzQl9rZtZOj6Q> . Me permitió tener una noción de como entender el diagrama de entidad relacion.

<https://www.youtube.com/watch?v=Wr-z7dOzUwE> . Este video me ayudó a tener más claro lo que es un diagrama de clase de diseño.

prompt para la ia : conviérteme este html con funciones de django a html puro, pero que sirvan los modals de la vista administrador, además hazme reservas llenas para así tener una demo funcional. este prompt lo realice para reutilizar un html que tenía de una aplicación igual que la usaba en django, este prompt lo use para el mockup.

Prompt para la ia: Quiero que me expliques como hacer un testing de caja blanca y de caja negra. Se usó para la realización del test de caja negra y caja blanca .

Prompt para la ia :

Como hacer el test de junit teniendo el caso de uso de la función realizar reserva. Me ayudó a hacer correctamente el testing con junit, además de corregir errores que tenía.

https://www.youtube.com/watch?v=_IxuUBDNrvi .Se utilizo para realizar el diagrama de clase de analisis

prompt para la ia: Explicame sobre diagrama de clase de análisis. este prompt me complemento la información para realizar el diagrama de clase de análisis.